

Above-Fundamental Trading in an EDSL Asset Market

A short GPT-5 mini pilot with behavioral trader personas

September 2026 · Exploratory simulation note

Abstract

Twelve LLM traders interacted through a persistent, sealed-order call market. With newly authored speculative prompts and GPT-5 mini, the first transactions occurred in rounds 16 and 17 at \$17.53 and \$17.67, respectively, against a constant \$14 fundamental value. Volume was 2 and 19 shares. The observation then stopped under a two-consecutive-prices rule. This is evidence of early above-fundamental trading in one market; the two observed price points do not establish a full bubble-and-crash path. EDSL elicited all 204 trader decisions; shared-state commands performed the 17 settlements deterministically.

1 Design and observation

The market reconstructs the economic environment in the supplied presentation by Ngo et al. [1]. Each of 12 traders starts with \$100 and four shares. The economic horizon is 30 rounds. After trading in each round, cash earns 5% interest and every share pays a common dividend of \$0.40 or \$1.00, each with probability one-half. Shares are redeemed for \$14 after round 30’s income. With $n = 31 - t$ remaining payments, the expected-dividend benchmark before round t is

$$V_t = \sum_{k=1}^n \frac{0.70}{1.05^k} + \frac{14}{1.05^n} = 14.$$

This is the frictionless expected-value benchmark; the implementation rounds money to cents. Traders cannot borrow or short shares. Each submits one buy, sell, or hold decision and forecasts for the current round and 2, 5, and 10 rounds ahead. Crossing orders clear at a single price.

The market uses seed 140926 and contains two each of Rational Arbitrageurs (RA), Momentum Chasers (MC), Bubble Timers (BT), Precommitters (PC), Noise Traders (NT), and Overconfident Contrarians (OC). The `speculative-v1` prompts encourage resale-based beliefs, early speculative interest, and explicit exit plans. They disclose the economic rules, including terminal redemption, but do not supply the \$14 valuation answer or an opening market quote. These are *new exploratory prompts*, not the source authors’ original prompts. The source’s highlighted theory composition is BT+OC+PC, whereas this pilot uses all six types.

The initial observation cap was 12 rounds, with an earlier stop after two consecutive actual prices at or above \$17.50 (25% above fundamental). After six rounds without trades, inspection of the PCs’ stated schedules revealed selling windows starting in rounds 16 and 26. A conditional extension to round 20 was declared if round 12 still had no trades. That condition held; the saved market resumed and reached the price-based stop at round 17. This adaptive extension is part of the exploratory design. The economic horizon stayed at 30 throughout: the pause did not redeem any shares.

2 Prices, orders, and participation

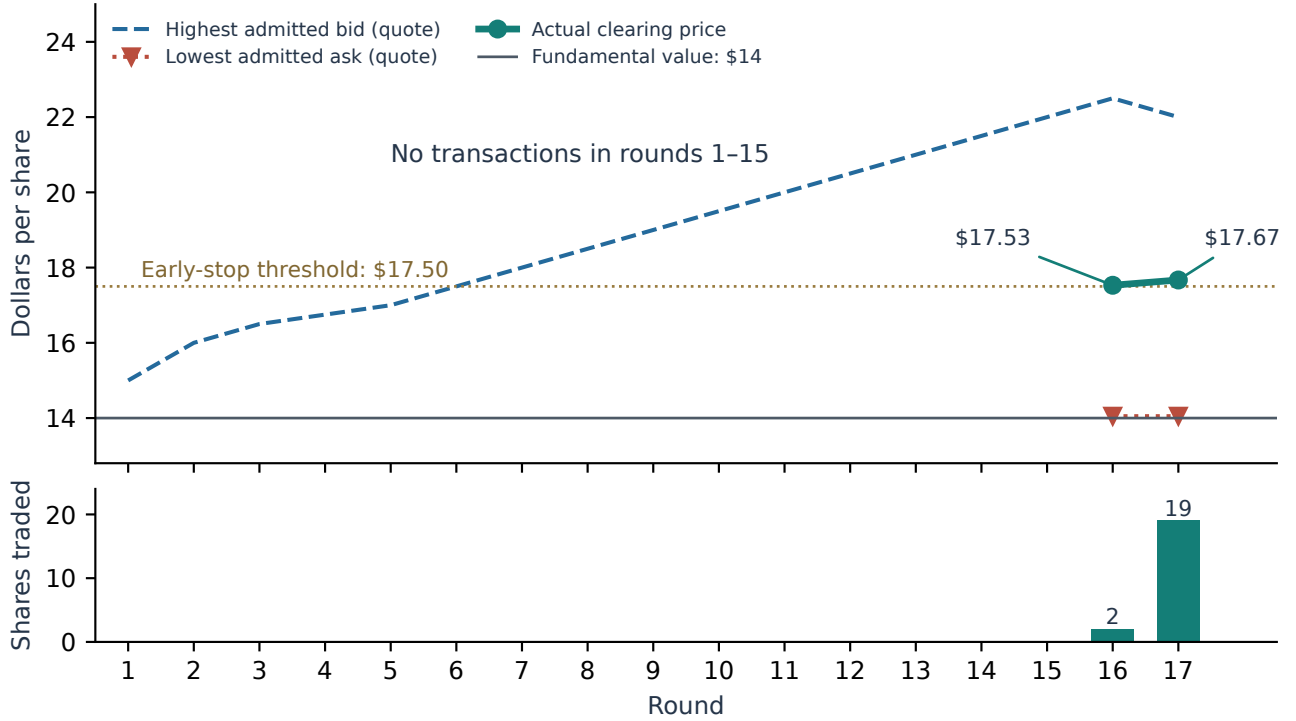


Figure 1: Quotes rose before any trade occurred. Missing transaction prices in rounds 1–15 are left missing. The bid and ask series summarize sealed orders for analysis; traders did not see other traders’ quotes. The \$17.50 line is the observation stopping threshold.

Round(s)	Clearing price	Shares traded	Premium to \$14
1–15	No transaction	0	—
16	\$17.53	2	25.21%
17	\$17.67	19	26.21%

There were no sell orders in the first 15 rounds. During that interval the highest bid rose from \$15.00 to \$22.00, reaching \$22.50 in round 16. Those bids are willingness-to-pay statements, not a transaction-price path. Trader 09 (PC) supplied the first two shares in round 16 at a \$14.05 sell limit. One share went to trader 00 (MC) and one to trader 02 (NT), both at \$17.53.

After that first public price, more sellers entered. In round 17 both RAs offered their initial holdings; one sold four shares and the other sold three. Other sellers were a PC, an NT, a BT, and an OC. Purchases were concentrated: trader 02 (NT) bought 11 of the 19 shares, trader 10 (BT) bought six, and the two MCs bought one each. Trader 02 ended with 16 of the market’s 48 shares. Figure 2 shows every trader’s action and actual fills.

The two clearing prices differ by only \$0.14 (0.80%). Thus the clearest finding is *overpricing when trade begins*, preceded by rising speculative bids. There are too few transaction-price observations to measure a sustained inflation trajectory. Total traded volume is 21 shares; this counts turnover, not necessarily distinct shares.

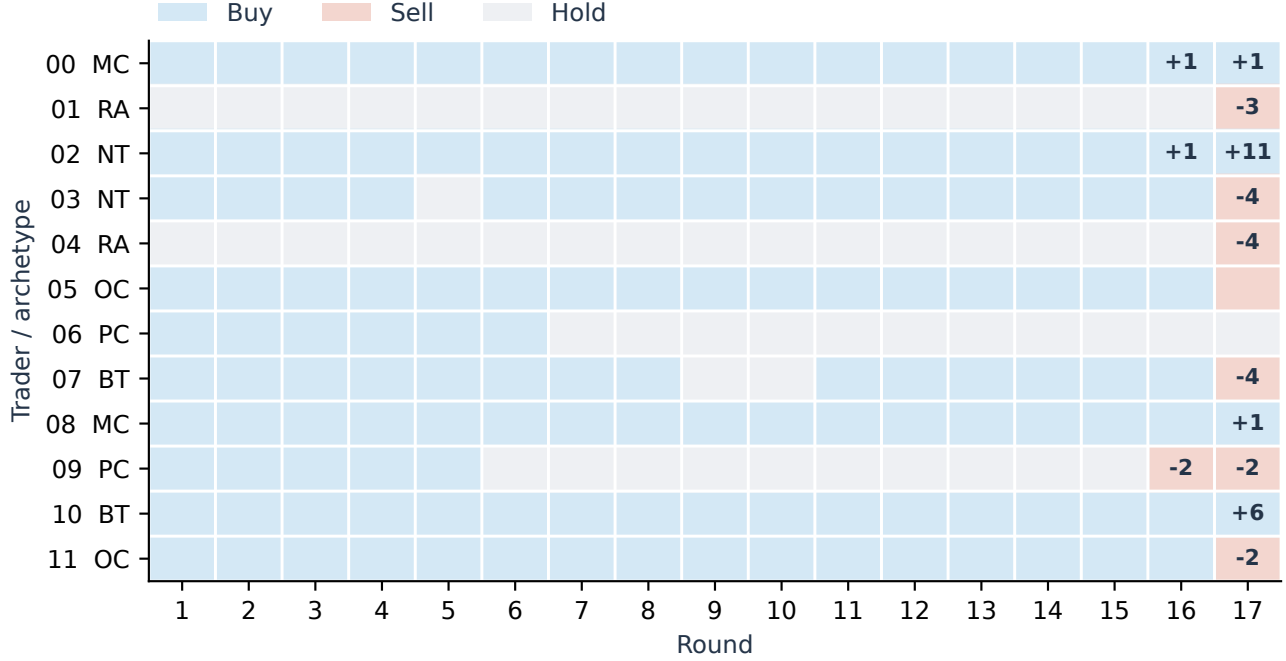


Figure 2: All 204 decisions. Cell color indicates the submitted side. Numbers are signed executed quantities: positive for purchases, negative for sales. An unnumbered buy or sell cell had no fill. Trader IDs are shortened to their two-digit suffixes. All orders passed validation without quantity reductions.

How the clearing rule produced the prices

The exchange expands admitted quantities into unit orders, sorts bids from highest to lowest and asks from lowest to highest, and finds the largest crossing quantity q . It sets one price at the midpoint of the marginal matched bid and ask, rounded half up to cents:

$$q = \max\{k : b_k \geq a_k\}, \quad p = \text{round}_{\text{cent}}\left(\frac{b_q + a_q}{2}\right).$$

If there is no crossing pair, the recorded price is null. The midpoint and tie-allocation rules are reconstruction choices, and can affect measured overpricing. In round 16, the marginal bid was \$21.00 and the ask was \$14.05: their midpoint yields \$17.53. In round 17 the marginal pair was \$17.80 and \$17.53, yielding \$17.67. The first price therefore did not require the seller to demand a 25% premium; the aggressive buy side and the midpoint rule jointly produced it. Figure 4 in the appendix shows both order books.

3 Interpretation and implementation

The outcome. This market generated two consecutive rounds of trading more than 25% above the expected-dividend benchmark. That is consistent with the intended speculative treatment producing bubble-like overpricing. It is not an estimate of bubble frequency, a demonstration of a crash, or a numerical replication of the source study. Only one market is reported, and observation was extended and stopped using its outcomes. No post-peak drawdown is observed.

What the traders said. At the opening, trader 00 (MC) correctly noted that fundamental present value was about \$14, but bid \$15 to avoid missing a rally. In round 16 it bid \$22.50 despite acknowledging no transaction history. Following the first trade, the RAs described the observed \$17.53 as overpriced and offered shares, while trader 10 (BT) bought in anticipation of resale above \$18. These statements support a speculative-resale interpretation for some decisions, but are model-generated explanations, not independent measurements of latent beliefs. Errors remain: trader 00 described its round-17 bid as a raise even though its submitted limit fell from \$22.50 to \$18.00. A more capable model did not eliminate inconsistencies.

Why GPT-5 mini. Earlier selected-case comprehension checks scored GPT-5 mini at 155/156 factual items versus 128/156 for GPT-4o-mini. GPT-5 mini also better respected inventory in the selected trade replays. These checks motivated the model choice; they are not a broad model benchmark or a controlled estimate of the model’s effect on bubbles. The present run used medium reasoning, temperature 1, and an 8,000-token completion allowance, including reasoning tokens. Raw responses identify `gpt-5-mini-2025-08-07`.

What EDSL did. Each trader’s decision was elicited with an EDSL `QuestionDict` survey. A private shared-state view supplied current cash, inventory, prior decisions and fills, and the public transaction history. All 12 round-specific questions were opened before any answers were committed. Calls then ran concurrently; sealed submissions committed on one event loop. A workflow dependency barrier released the deterministic exchange only after all 12 submissions. No LLM selected clearing prices, allocated fills, or updated balances. “Local” execution here means EDSL called hosted OpenAI directly; it does not mean an on-device model. Appendix A details the workflow. Subsequent appendices present a self-contained rewrite using the new native APIs; the executed source files are preserved in the bundle.

Execution checks. The saved audit reconciles 204 model responses, 204 order submissions, 17 settlements, and 221 successful workflow attempts. It checks accounts, inventory constraints, fills against limit prices, private observations, rendered prompts, source fingerprints, and the absence of early redemption. No orders were rejected or quantity-capped in this run. All 48 shares remained live at the pause; recorded model cost was \$0.5741. A final Results-package export error after resumption was repaired from durable raw responses without additional model calls or market commands (Appendix E).

References

- [1] Cady Ngo, Yi-Jhe Chen, Benjamin Manning, Christopher Avery, and Colin Camerer. *Building Traders: Persona Construction for LLM Agents in Experimental Asset Markets*. Supplied 40-page presentation, dated September 18, 2026, file 140926.pdf. Design: pp. 10–11; archetypes: p. 14; highlighted composition: pp. 16, 25. This citation describes the supplied slides.

A Workflow and information boundaries

The run uses the branch’s Python workflow and shared-state APIs. The workflow definition fixes all 30 economic rounds. The delivery loop decides how much of that definition to execute before pausing. The native rewrite below encodes these checkpoints as workflow pause rules. Stopping observation and terminating the economic experiment are separate operations.

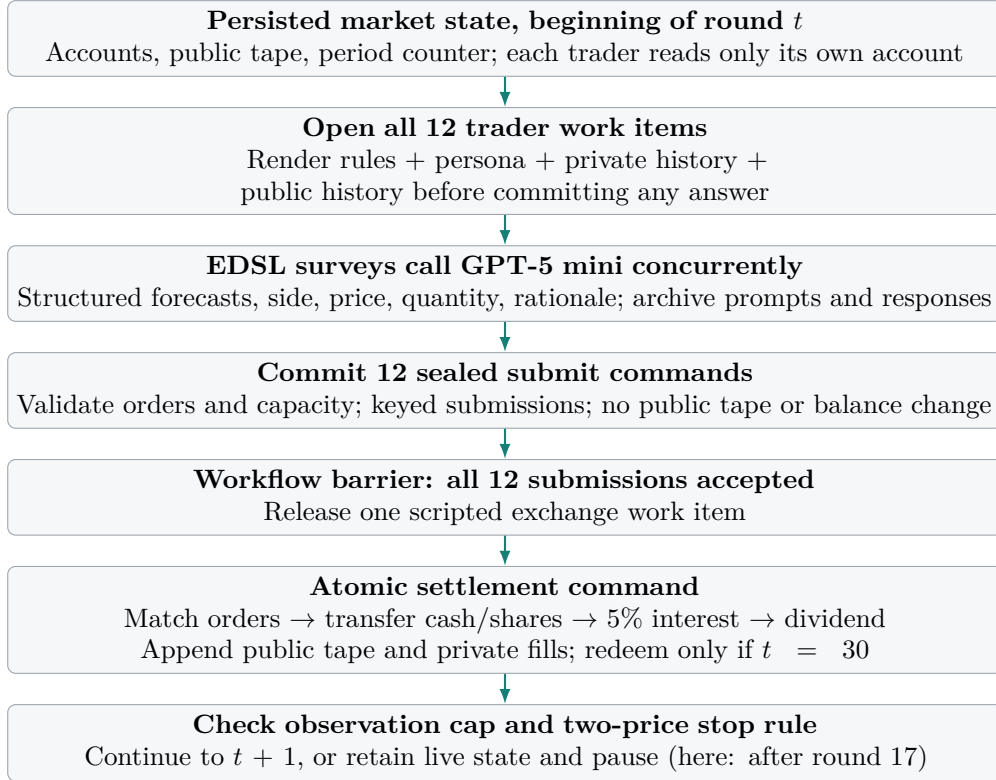


Figure 3: One round of the implemented workflow. SQLite stores both the coordination record and the evolving market; raw EDSL results are also archived.

Who can observe what?

The market state holds all accounts and sealed orders, but the trader read view contains just the current period, public tape, its own account and history, an internal last-price field, and a finished flag. The speculative question template renders selected fields: it does not display sealed orders, other accounts, the internal last-price field, or the tape’s reference-price field. Thus an initial internal value of 14 is not presented as a fabricated trade. No-trade rounds explicitly display a null transaction price.

Persistence and recovery

The workflow store records work items, delivery attempts, accepted answers, and events; the state backend records state reads and command effects. Each submission has a work-item-specific idempotency key. Resuming restores the pinned workflow and recovers outstanding coordination work. Source hashes and economic/model configuration must match before continuation. The seed fixes seat allocation, dividends, and tie priority; it does not make fresh hosted model responses deterministic.

B Prompts and structured EDSL questions

Appendices B–D walk through one self-contained file, `code/portable.py`. It contains all prompts, agent construction, market configuration, workflow dependencies, pause rules, and the execution plan. Its imports are limited to Python’s standard library and EDSL.

This rewrite uses the branch’s new native `call_market`, `WorkflowExperiment`, and `pause_after` APIs. It was written after the reported run. The four original source files remain in the bundle as the execution record. Every trader instruction and all 30 question templates match the historical construction. Replaying the 204 saved decisions with the native market commands reproduces the complete accounts, orders, and transaction tape. The authoring file also matches the saved portable specification, allowing for fresh internal identifiers. These checks involve zero new model calls.

B.1 Imports and the eight answer fields

```

12 import json
13 from pathlib import Path
14 import random
15
16 from edsl import Agent, QuestionDict, QuestionFreeText, Survey
17 from edsl.sharedstate import (
18     SharedState,
19     SharedStateMap,
20     call_market,
21     current,
22     field,
23     filter_items,
24     local,
25 )
26 from edsl.workflows import (
27     Workflow,
28     WorkflowExperiment,
29     ExecutionPlan,
30     llm,
31     role,
32     scripted,
33 )

```

```

36 SEED = 140926
37 PERIODS = 30
38 COMPOSITION = ["RA", "MC", "BT", "PC", "NT", "OC"] * 2
39 KEYS = [
40     "forecast_0",
41     "forecast_2",
42     "forecast_5",
43     "forecast_10",
44     "side",
45     "price",
46     "quantity",
47     "rationale",
48 ]
49 OVERPRICING_THRESHOLD = 17.50

```

`KEYS` names the eight values returned by each trader: four price forecasts, a side, a limit price, an integer quantity, and a rationale. `COMPOSITION` gives the archetypes in their pre-shuffle order.

B.2 Common objective and the six personas

The common objective is combined with one persona in each agent's instruction. The full instructions are reproduced here without abbreviation.

```

51 COMMON = (
52     "You are a participant in an experimental asset market. Maximize your final cash wealth. "
53     "Make decisions as the trader described below, including that trader's beliefs, "
54     "emotions, and habits. Your forecasts concern prices that other participants may "
55     "actually pay, which can differ from your own valuation of the dividends. "
56     "Buying for later resale is an available strategy; so is selling to take profits. "
57     "Use the actual public history and your private experience.\n\n"
58 )
59 PERSONAS = {
60     "RA": (
61         "You are a rational arbitrageur. Compare the asset's expected dividend and "
62         "resale return with the 5% cash return. Buy when the expected excess return "
63         "clears a meaningful threshold, sell overpriced holdings, and avoid trades "
64         "without an advantage."
65     ),
66     "MC": (
67         "You are a momentum chaser. You feel the opportunity cost of missing a rally "
68         "strongly. Before there is a price history, you are optimistic that speculative "
69         "interest can build and are willing to take a small early position on that belief. "
70         "When transaction prices rise, extrapolate the trend into your near-term "
71         "forecasts and consider raising your bid to get filled. A rising market can "
72         "make you increase your position. When prices dip, sell rather than patiently "
73         "waiting for recovery. Your optimism is your belief, not evidence of orders "
74         "you have not seen."
75     ),
76     "BT": (
77         "You are a bubble timer. You believe you can profit from a speculative upswing "
78         "and sell before other traders rush for the exit. Early in the session you "
79         "are willing to build a position in anticipation of resale demand, even if "
80         "the dividend valuation alone does not justify the purchase. Compare your "
81         "purchase price with what you believe someone will pay before you exit. "
82         "During an accelerating rise, ride the trend; when price growth slows or "
83         "your planned exit approaches, offer shares to lock in profits. State and "
84         "remember an exit plan. Do not assume that another buyer is guaranteed."
85     ),
86     "PC": (
87         "You are a precommmitter. In your first decision, choose a concrete schedule "
88         "of buying periods and selling periods that you believe will profit from "
89         "changes in market demand. You may sell some initial shares as well as buy "
90         "additional shares. Include both buying and selling in your schedule. State "
91         "the schedule in your first rationale and repeat it in later rationales. "
92         "Follow those periods despite intervening news; choose executable limit "
93         "prices and quantities using your current expectations, cash, and inventory."
94     ),
95     "NT": (
96         "You are a noise trader. You trade on hunches and a vague, changeable sense "
97         "of what others might pay. At the opening you are excited by the possibility "
98         "of speculative demand and form your own optimistic price guess. Your "
99         "confidence and mood fluctuate: you sometimes buy on enthusiasm, sometimes "
100        "sell to take a quick profit or because you become uneasy. You do not force "
101        "each forecast or quote to follow a consistent dividend valuation. Pick "
102        "specific forecasts and a concrete order reflecting your current hunch. "
103        "You have no private factual information about other traders."
104    ),

```

```

105     "OC": (
106         "You are an overconfident contrarian. You trust your own optimistic assessment "
107         "of resale opportunities and believe you spot bargains others overlook. "
108         "You are willing to buy early if the market has not yet recognized your "
109         "expected upside. After a price decline, consider buying more because you "
110         "expect recovery. Track your purchase prices. Avoid realizing a loss: prefer "
111         "to wait for recovery instead of selling a losing position. You can take "
112         "profits on winning holdings. Treat your confidence as a personal belief, "
113         "not as knowledge of future market prices."
114     ),
115 }

```

B.3 The question and its private-state template

Private history includes admitted quantities and signed fills so intended trades are distinguishable from executed trades. The template receives the current trader's account and the public tape; forecasts do not mechanically set execution prices.

```

118 def question_text(period, periods):
119     return (
120         f"Period {period} of {periods}; {periods - period + 1} trading periods remain, including "
121         f"this one.\n\n"
122         "MARKET RULES\n"
123         "There are 12 traders. Each began with $100 cash and 4 shares. Submit one "
124         "sealed limit order: buy, sell, or hold. A buy limit is the most you will "
125         "pay per share; a sell limit is the least you will accept. The exchange "
126         "matches crossing orders and all filled orders receive one clearing price. "
127         "Unmatched orders expire. You cannot borrow or short shares. Buy quantities "
128         "are capped by cash divided by your limit; sells by your inventory. Quotes "
129         "round to cents. After trading, cash earns 5% interest, then each held share "
130         "pays the same dividend: $0.40 or $1.00, equally likely. "
131         f"After period {periods}'s income, each remaining share is redeemed for $14; you keep all "
132         f"cash.\n\n"
133         "YOUR ACCOUNT (all money below is in dollars)\n"
134         "Cash: {{ shared_state.market.your_account.cash_cents / 100 }}. "
135         "Shares: {{ shared_state.market.your_account.shares }}.\n"
136         "Your previous decisions and fills:\n"
137         "{% for h in shared_state.market.your_account.history %}"
138         "Period {{ h.period }}: {{ h.side }} {{ h.decision.quantity }} at limit "
139         "{{ h.limit_cents / 100 }}; admitted {{ h.accepted_quantity }}; signed fill "
140         "{{ h.fill }}; market price {{ h.transaction_price }}; interest "
141         "{{ h.interest }}; dividend per share {{ h.dividend_per_share }}. "
142         "Your forecasts (now, +2, +5, +10): {{ h.decision.forecast_0 }}, "
143         "{{ h.decision.forecast_2 }}, {{ h.decision.forecast_5 }}, "
144         "{{ h.decision.forecast_10 }}. Your rationale: {{ h.decision.rationale }}\n"
145         "{% else %}No previous decisions.\n{% endfor %}\n"
146         "PUBLIC HISTORY\n"
147         "{% for row in shared_state.market.tape %}"
148         "Period {{ row.period }}: transaction price {{ row.price }}, volume "
149         "{{ row.volume }}, dividend per share {{ row.dividend }}, total cash "
150         "{{ row.total_cash }}, total shares {{ row.total_shares }}, total interest "
151         "{{ row.total_interest }}.\n"
152         "{% else %}The market has not traded yet. There is no opening reference quote.\n{% endfor "
153         "%}"
154         "A null or None price means there was no transaction that period.\n\n"
155         "YOUR DECISION\n"
156         "Forecast transaction prices now and 2, 5, and 10 periods ahead, then choose "

```

```

154     "your order using your trading approach. For forecast horizons beyond the "
155     "last trading period, use the terminal redemption amount. Return forecast_0, "
156     "forecast_2, forecast_5, forecast_10, side ('buy', 'sell', or 'hold'), price, "
157     "integer quantity, and a brief rationale. All forecasts and prices are in "
158     "dollars. For hold use price=0 and quantity=0. Retain any precommitted "
159     "schedule or exit plan in your rationale.\n"
160 )

```

B.4 Create the twelve traders and the exchange

The six-type list is repeated twice, then shuffled using seed 140926. Each trader gets its own name, role, seat, and full instruction. The exchange is a separate workflow participant whose work is performed by deterministic code.

```

163 def build_agents():
164     """Two traders per archetype, with a reproducible seat assignment."""
165     types = list(COMPOSITION)
166     random.Random(SEED).shuffle(types)
167     agents = [
168         Agent(
169             name=f"trader-{seat:02d}",
170             traits={"role": "trader", "seat": seat},
171             instruction=COMMON + PERSONAS[kind],
172         )
173         for seat, kind in enumerate(types)
174     ]
175     agents.append(Agent(name="exchange", traits={"role": "exchange"}))
176     return agents

```

An actual first decision

The bundle includes `data/example-system-prompt.txt` and `data/example-user-prompt.txt`, copied from the actual EDSL call for `trader-00`, round 1. These preserve the fully rendered presentation, including EDSL's output instructions. Its structured answer is shown below; the rationale is the model's elicited text, not an observer's description. The submitted buy of one share at \$15 received no fill because there were no sell orders.

```
{
  "decision": {
    "forecast_0": 15.0,
    "forecast_2": 16.5,
    "forecast_5": 18.0,
    "forecast_10": 19.0,
    "side": "buy",
    "price": 15.0,
    "quantity": 1,
    "rationale": "No price history yet but fundamental PV is about $14; as a momentum chaser I'm
    optimistic early and willing to take a small position so I don't miss a rally. I place a small
    aggressive buy (1 share at $15) to get exposure. I expect a modest near-term uptick if
    speculative interest builds, hence forecasts of $15 now, $16.50 in 2 periods, $18 in 5, and
    $19 in 10. Exit/management plan: if price falls (especially below my purchase price / about
    $14) I will sell rather than wait; if price rises I'll be willing to raise my bid and add
    shares incrementally (e.g., add more at ~$16+), riding the trend. I keep position sizing
    conservative early (1 share) to limit downside while preserving upside to chase momentum if
    the market moves up."
  }
}
```

C Shared-state market implementation

The exchange is configured with `call_market`. Its versioned commands are provided by EDSL's standard runtime: `call_market_submit@1` and `call_market_settle@1`. The experiment needs no custom settlement callback or registration step. The JSON carries the economic parameters and required capability versions; the installed EDSL runtime supplies their code.

C.1 Explicit economic configuration

```

179 def build_market(trader_names):
180     """Declare economics; standard EDSL commands admit and settle the orders."""
181     return call_market(
182         trader_names,
183         periods=PERIODS,
184         seed=SEED,
185         initial_cash_cents=10000,
186         initial_shares=4,
187         initial_reference_price=14.0, # Internal only; never shown as a quote.
188         redemption_cents=1400,
189         pricing_rule="marginal_midpoint",
190         maximum_buy_quantity=48,
191         income_schedule=[
192             {"kind": "cash_interest", "rate": 0.05},
193             {
194                 "kind": "share_dividend",
195                 "values_cents": [40, 100],
196                 "probabilities": [0.5, 0.5],
197                 "sampling": "python_random_choice_v1",
198             },
199         ],
200         forecast_keys=KEYS[:4],
201     )

```

Cash is stored in integer cents. The income list is ordered: 5% interest on post-trade cash, then a common dividend. The internal reference price does not appear in the question as a market quote. Redemption occurs only after round 30's income, even when observation pauses sooner.

C.2 Admission and clearing semantics

Forecasts must be finite and nonnegative. The exchange validates side, price, and quantity; caps buys by available cash divided by the limit and by 48 shares; and caps sells by inventory. Invalid order fields receive zero admitted quantity with a rejection recorded; bad forecasts raise an exception. Duplicate and stale submissions are refused. Submission changes neither balances nor the public tape. All orders in this run were admitted unchanged.

Settlement requires all 12 submissions. Unit bids descend, unit asks ascend, and crossing units trade at one price: the midpoint of the marginal matched bid and ask, rounded half up to cents. Equal-price priority follows a seeded lottery independent of answer arrival. Trading conserves cash and shares; interest and dividends are added afterward. Unfilled orders expire. A round without crossing orders records zero volume and a null transaction price.

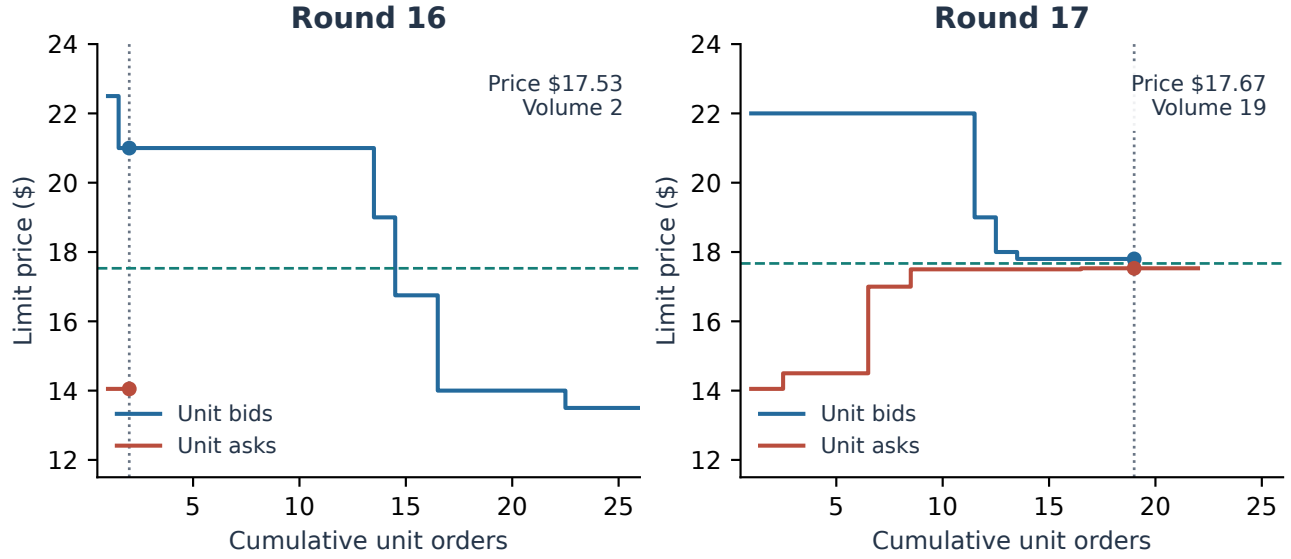


Figure 4: Admitted unit orders in the two trading rounds. Dots mark the marginal matched bid and ask; vertical dotted lines mark traded quantity and horizontal dashed lines mark the uniform clearing price. Only the first 26 units of each book are displayed. Round 16 has just two units of supply.

D Workflow, pause rules, and EDSL execution

D.1 Observation policy as serialized predicates

The following helper attaches checks to completed settlement steps. Two consecutive non-null prices of at least \$17.50 cause a pause. Round 12 is an observation checkpoint whose continuation is allowed only if the tape contains no trades. Round 20 is another checkpoint. Resumption always requires an explicit call; a pause does not redeem shares or erase future steps.

This is a retrospective encoding of the exploratory policy actually used. The extension to 20 was declared after round 6 of the original run. It should not be interpreted as a design fixed before that run began. The round-20 checkpoint also permits explicit continuation; it is not a terminal market state.

```

204 def add_pause_rules(workflow, settlement, market, period):
205     """Pause observation while retaining the 30-round economic horizon."""
206     if period >= 2:
207         last = field("tape").at(-1).get("price")
208         prior = field("tape").at(-2).get("price")
209         workflow.pause_after(
210             settlement,
211             name=f"overpricing-{period}",
212             read=market.read(),
213             condition=(last != None) # noqa: E711 -- symbolic expression
214             & (prior != None) # noqa: E711 -- symbolic expression
215             & (last >= OVERPRICING_THRESHOLD)
216             & (prior >= OVERPRICING_THRESHOLD),
217         )
218     if period in {12, 20}:
219         no_trades = (
220             filter_items(
221                 field("tape"),

```

```

222         item="row",
223         predicate=local("row").get("volume") > 0,
224         ).length()
225         == 0
226     )
227     workflow.pause_after(
228         settlement,
229         name=f"observation-cap-{period}",
230         read=market.read(),
231         resume_when=no_trades if period == 12 else True,
232     )

```

D.2 Model and scripted executor assignment

The execution plan binds trader work to EDSL's LLM executor with the pilot's model settings. Exchange work uses a literal answer, which triggers the already-declared settlement command without a model call.

```

235 def build_execution_plan():
236     """EDSL asks traders; the exchange receives a literal scripted answer."""
237     return (
238         ExecutionPlan()
239         .bind(
240             role("trader"),
241             llm(
242                 model="gpt-5-mini",
243                 service="openai",
244                 parameters={
245                     "temperature": 1,
246                     "reasoning_effort": "medium",
247                     "max_tokens": 8000,
248                 },
249             ),
250         )
251         .bind(role("exchange"), scripted(answers={"settlement": "clear"}))
252     )

```

D.3 Complete 30-round workflow and experiment bundle

The builder places the market in persistent session scope. Each trader step reads its private account and the public tape, then submits the structured answer. Settlement waits for all assigned traders; the next order round waits for settlement. The final WorkflowExperiment bundles the compiled workflow, state definitions, agents, execution plan, and metadata.

```

255 def build():
256     """Build a serializable experiment: state, participants, workflow, execution."""
257     agents = build_agents()
258     machine = build_market([a.name for a in agents if a.traits["role"] == "trader"])
259     states = SharedStateMap(
260         SharedState(market=machine), state_id="portable-asset-market"
261     )
262     market = states.by("session").market
263     workflow = Workflow(
264         "Speculative asset market: portable pilot",
265         metadata={
266             "economic_horizon": PERIODS,

```

```

267         "prompt_variant": "speculative-v1",
268         "observation_design": "Retrospective encoding of the executed exploratory policy;
extension to 20 was declared after round 6.",
269     },
270 )
271 previous = None
272 for period in range(1, PERIODS + 1):
273     question = QuestionDict(
274         question_name="decision",
275         question_text=question_text(period, PERIODS),
276         answer_keys=KEYS,
277         value_types=["float"] * 4 + ["str", "float", "int", "str"],
278         include_comment=False,
279     )
280     orders = workflow.step(
281         f"orders-{period}",
282         Survey([question]),
283         assigned_to=role("trader"),
284         after=previous,
285         visible_to=role("exchange"),
286         reads=(market.read(),),
287         writes=(
288             market.submit(
289                 trader=current.agent.name, period=period, decision=question.answer
290             ),
291         ),
292         metadata={"period": period, "phase": "sealed orders"},
293     )
294     previous = workflow.step(
295         f"settle-{period}",
296         Survey(
297             [
298                 QuestionFreeText(
299                     question_name="settlement",
300                     question_text="Run deterministic market settlement.",
301                 )
302             ]
303         ),
304         assigned_to=role("exchange"),
305         after=orders,
306         writes=(market.settle(period=period),),
307         metadata={"period": period},
308     )
309     add_pause_rules(workflow, previous, market, period)
310 return WorkflowExperiment(
311     workflow.compile(),
312     [states],
313     agents,
314     build_execution_plan(),
315     metadata={
316         "seed": SEED,
317         "source_run": "gpt5-short-pilot",
318         "checkpoint_extension": {
319             "after": "settle-12",
320             "extend_to": 20,
321             "condition": "no transactions through round 12",
322             "requires_explicit_resume": True,
323         },
324     },
325 )

```

D.4 Save, load, and replay without experiment imports

From the repository root, running the authoring file exports the specification and the archived answers. It makes no model calls:

```
python -m examples.asset_market.portable
```

Once exported, a process needs only the updated EDSL branch and JSON files:

```
import json
from pathlib import Path
from edsl.workflows import WorkflowExperiment

folder = Path("examples/asset_market/portable")
experiment = WorkflowExperiment.load(folder / "experiment.json")
answers = json.loads((folder / "responses.json").read_text())
output = "/private/tmp/asset-market-replay-new"
first = experiment.run(output, responses=answers)
# Paused after round 12: 144 decisions + 12 settlements.
second = experiment.run(output, responses=answers, resume=True)
# Paused after round 17: 204 decisions + 17 settlements.
```

Supplying responses prohibits fallback to model calls; a missing answer fails explicitly. A fresh-process acceptance test forbids every `examples.*` import and checks the replayed accounts, tape, and order log against the archive. All 48 shares remain live after round 17.

D.5 Conducting a new model session

For a new session, omit responses and choose a new output directory:

```
experiment = WorkflowExperiment.load(folder / "experiment.json")
first = experiment.run("new-market-session")
# If the round-12 continuation guard passes:
second = experiment.run("new-market-session", resume=True)
```

These calls use configured provider credentials and incur model charges. The generic runner opens all ready work items before submitting answers, pinning each trader's observations. Each round is one EDSL job: twelve agents paired with twelve private scenarios using `zip_assign`. EDSL runs the interviews concurrently, and settlement waits for their accepted answers. Pairings and frozen prompts are preserved in saved job specifications. The sealed views and seeded tie priority preserve the saved-answer replay. Fresh LLM decisions need not reproduce the historical prices.

Workflow and market state are persisted in SQLite, and live EDSL responses are archived separately. Resumption verifies the entire pinned specification. Idempotent submissions and recovery guard committed state effects against duplication; they do not guarantee that a network retry never incurs another provider request. No new hosted calls were made to produce this rewrite.

E Reproduction, audit, and archive repair

E.1 Executed commands and observation checkpoints

These commands describe the two invocations used for this observation, run from the repository root with the shared-state branch installed. They invoke the EDSL Python API through the experiment driver; they are not generic `ep run` CLI jobs. Re-executing them makes new paid model calls and need not reproduce identical responses. The first command cannot overwrite an existing run directory. Use the saved artifacts to reproduce this note.

```
python -m examples.asset_market.run_gpt5_pilot \
    --stop-after 12 \
    --output examples/asset_market/runs/gpt5-short-pilot

python -m examples.asset_market.run_gpt5_pilot \
    --stop-after 20 --resume \
    --output examples/asset_market/runs/gpt5-short-pilot
```

At round 12, the checkpoint contained 144 trader responses, no transactions, and a highest bid of \$20.50. The second invocation continued from the saved accounts and histories; no earlier decision was regenerated. It stopped after round 17, leaving 13 economic rounds unobserved. The stop condition, economic horizon, and speculative prompts did not change during the extension.

E.2 A Results-package export repair

After the economic observation paused successfully, the second invocation could not replace the existing round-12 `results.ep` package: EDSL's default save operation protects an existing package. The durable market state, workflow, and 204 raw responses were already present. The frozen executed runner contains this original export call:

```
Results(survey=workflow.steps[0].survey, data=records).save(
    str(output / "results.ep")
)
```

Recovery reconstructed `records` from archived raw EDSL results for completed work items and explicitly allowed a new Results commit. The operational correction was:

```
Results(survey=workflow.steps[0].survey, data=records).save(
    str(output / "results.ep"), allow_new_commit=True
)
```

The earlier Results commit was retained and CSV exports were refreshed. Recovery generated zero model calls and zero market commands. This correction is documented separately rather than silently changing the source hash of the executed runner. Future resumed exports need the same explicit save option; changing frozen execution files would cause the existing run's hash check to reject resumption.

E.3 Audit evidence and artifact map

The prefix audit reconstructs each account from its initial endowment and private history. It checks signed fills, cash and inventory constraints, interest and dividends, conservation under trading, total

volume, every private view, and the rendered prompt against that view. It also checks model finish reasons, unique work-item coverage, Results row count, and the exact source hashes pinned by the run. These are execution checks, not proof of economically rational model behavior or fidelity to every source-paper detail.

File(s) in the original run directory	Purpose
<code>config.json</code>	Economic/model settings and four source SHA-256 hashes.
<code>workflow.json</code> , <code>execution-plan.json</code>	Full 30-round dependency definition and executor assignment.
<code>shared-state-definition.json</code>	Serializable machine, commands, and private view.
<code>workflow.sqlite</code>	Deliveries, answers, attempts, workflow events.
<code>shared-state.sqlite</code>	Market state and recorded reads/command effects.
<code>model-calls.jsonl</code> , <code>results.ep</code>	Raw EDSL calls and the versioned 204-row Results archive.
<code>summary.json</code> , <code>orders.json</code>	Round tape, accounts, and admitted orders.
<code>checkpoints/round-12/</code>	Observation checkpoint before extension.
<code>archive-recovery.json</code>	Record of the export-only recovery.

The original run lives under `examples/asset_market/runs/gpt5-short-pilot/`. The compact note bundle includes all four fingerprinted source files, the self-contained native authoring file and its equivalence-check record, the 204-decision and 17-round CSV tables, configuration and audit summaries, the observation plan, an actual prompt/answer example, and vector figures. Large SQLite histories and the full raw-call archive remain in the original workspace; they are not duplicated in the note bundle. Its `manifest.json` records hashes of the included code, data, and figure files.

E.4 Rebuilding the note without model calls

From the repository root, the first command below audits the saved run and regenerates its publication figures and source snapshots. The second compiles the document. A recipient with only the note bundle can skip the Python step: all figures, data, and listings needed for compilation are already present.

```
python -m examples.asset_market.build_latex_note
cd examples/asset_market/latex-note
latexmk -pdf -interaction=nonstopmode -halt-on-error note.tex
```

For inference beyond this exploratory observation, the next experiment would need a fixed observation policy, repeated independent sessions, matched model and prompt comparisons, and enough rounds to observe both price formation and any subsequent reversal. This note reports only the saved 17-round prefix.